Vadim Kolotenko
Haaga-Helia ammattikorkeakoulu
Linux-palvelimet - ICI003AS2A-3021

Raportti 2
8.10.2026

Intro

The assignment was to practice the Linux command line on my Debian virtual machine. I created a directory tree in my home directory, edited and restored files, and packed those files into archives. After that I searched text with grep, wc and pipes, read the GPL-2 license that comes with Debian, and installed btop and fastfetch.

Directories and files

I started in my home directory and made ~/practice. Inside it I made docs, images, backups and archive. In docs I created four empty files, notes1.txt through notes4.txt. tree reported 5 directories and 4 files.

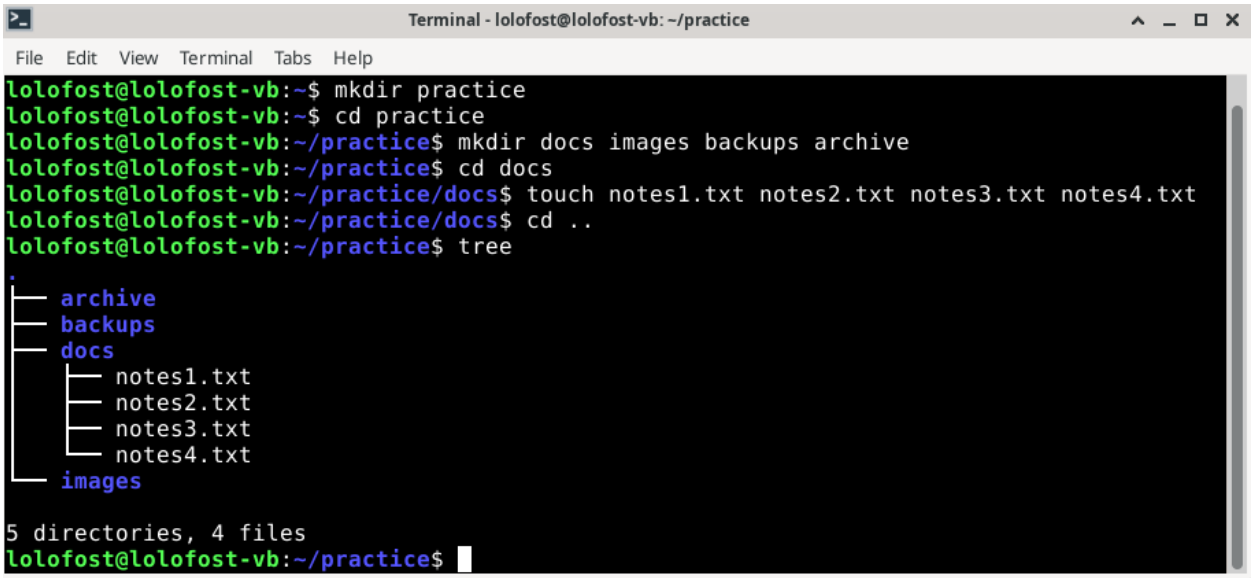


Image 1. The practice directory after it was created.

I edited the first two files in nano. notes1.txt got ten animals, and notes2.txt got ten vegetables and fruits. I renamed them with mv to animals.txt and vegetables.txt. notes3.txt and notes4.txt stayed empty.

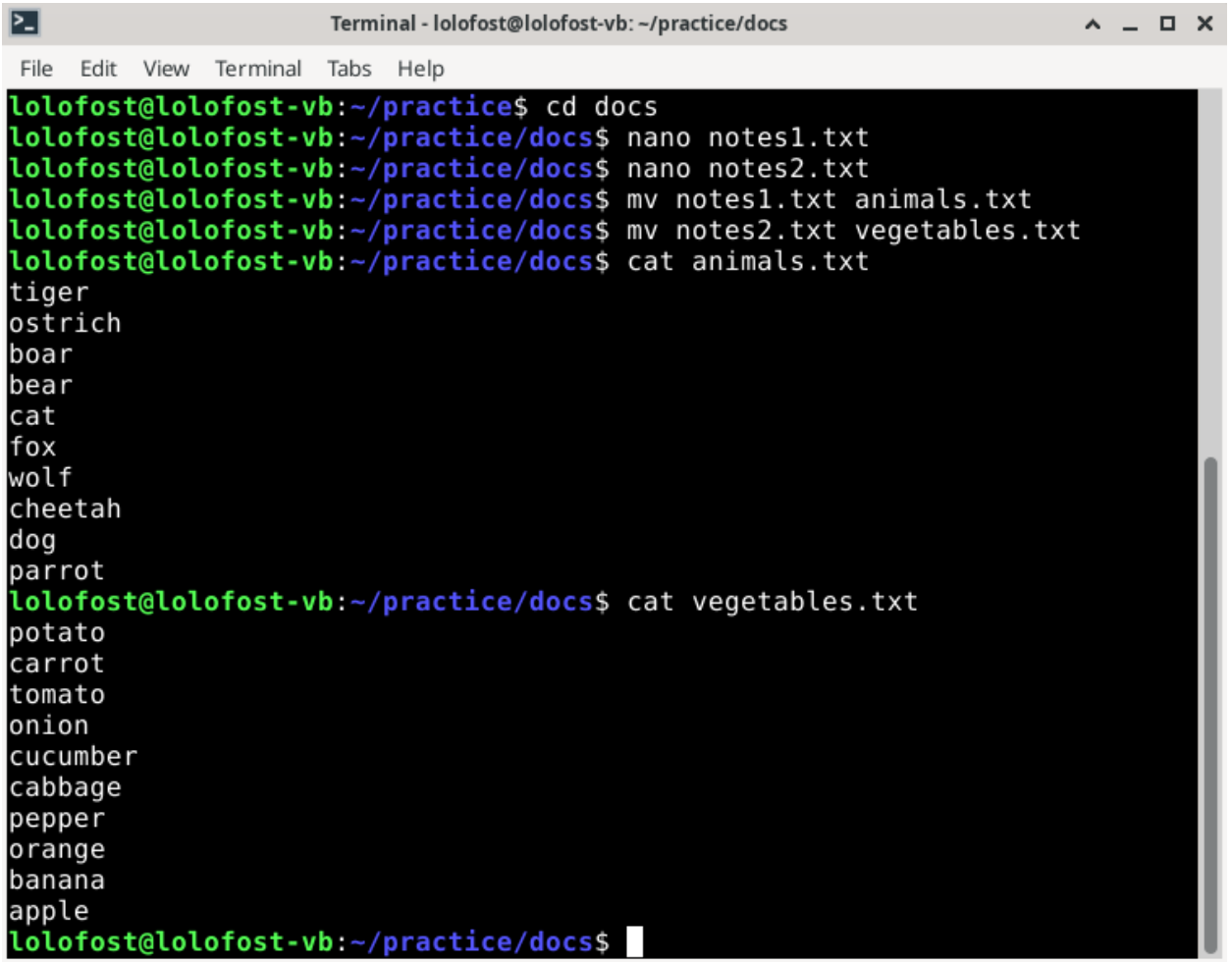


Image 2. The renamed files.

I copied both files into backups. Then I deleted cheetah, dog and parrot from animals.txt, and I removed vegetables.txt with rm. The copies in backups stayed as they were. In docs the animal file was down to 7 words, vegetables.txt was gone, and the two empty note files were still there.

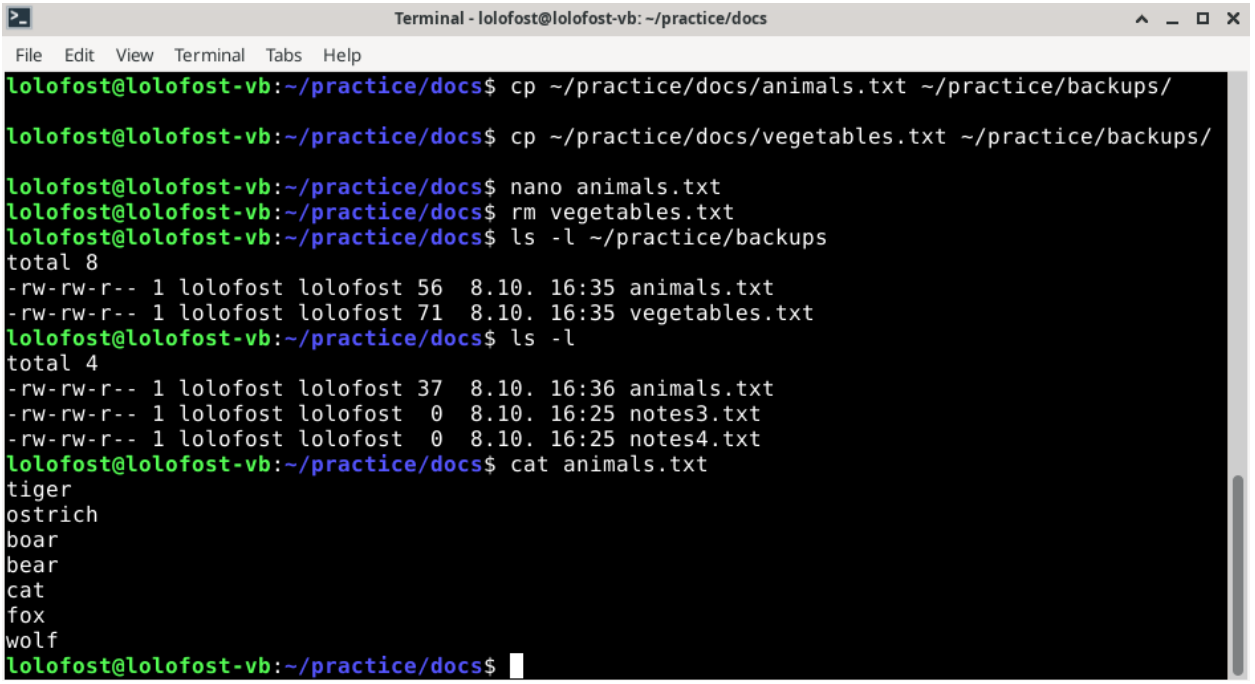


Image 3. Backups next to the edited docs directory.

I copied both files from backups back into docs. animals.txt listed all ten names again, including the three I had removed. The same copy restored vegetables.txt.

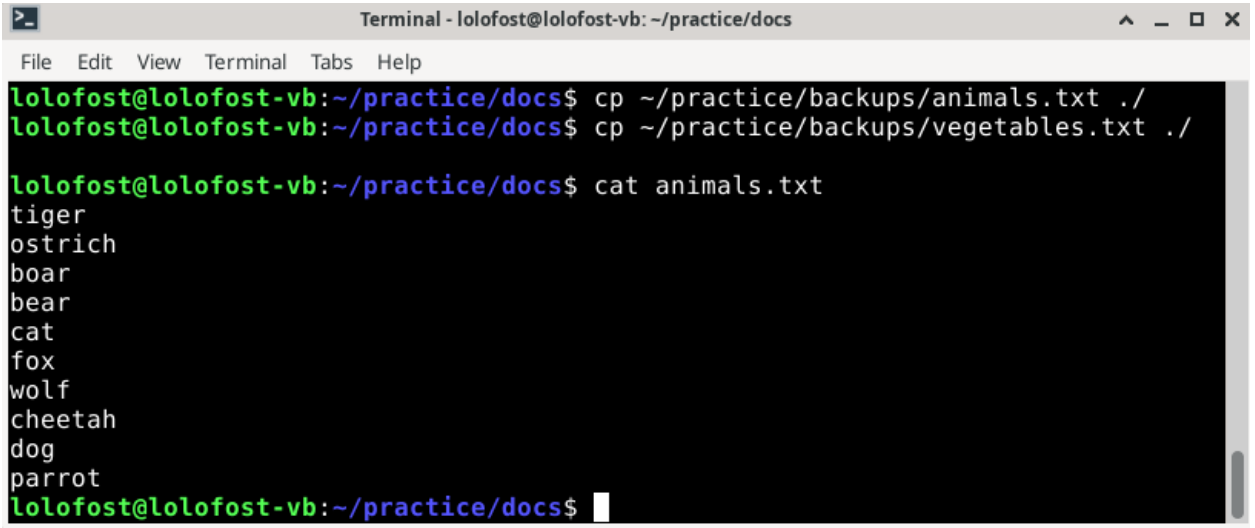


Image 4. animals.txt after the restore.

Archives

From ~/practice I packed the two backup files into archive/backups.tar. The option c creates the archive, v lists the files as they are added, and f gives the archive name. -C backups reads the files from that directory, so the archive stores animals.txt and vegetables.txt on their own. gzip then replaced backups.tar with backups.tar.gz.

The same two steps fit in one command. tar -czvf writes archive/backups-onestep.tar.gz directly, and z is the gzip step. I used a second filename so the first archive stayed as it was.

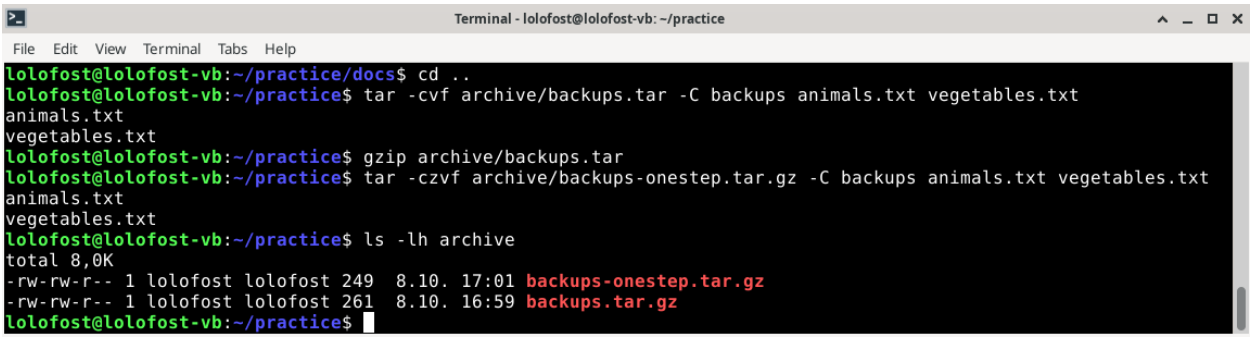


Image 5. Both compressed archives.

I made a directory called test and extracted backups.tar.gz into it with tar -xzvf. x extracts, and -C test chooses the destination. The two text files came out directly inside test.

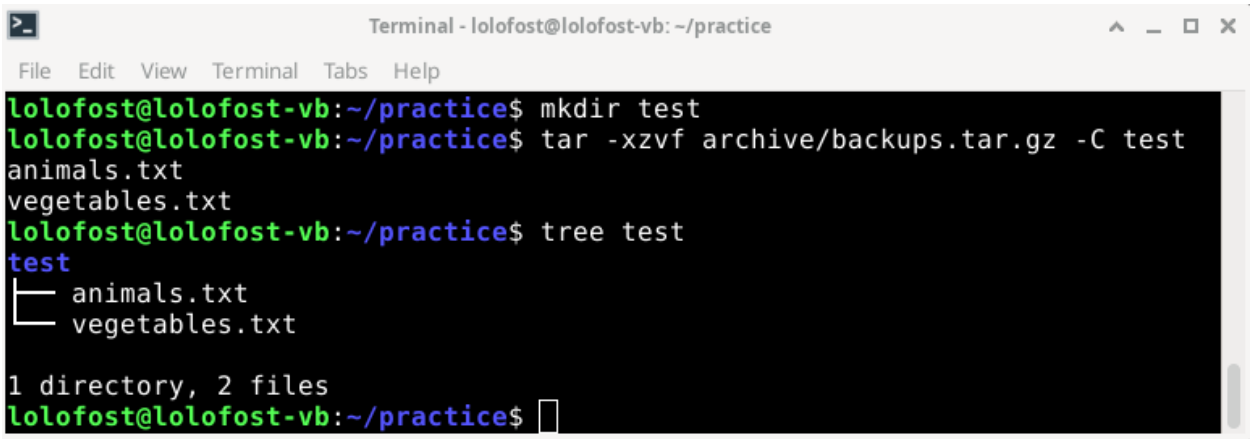


Image 6. The extracted archive.

grep, wc and pipes

I kept this part in ~/practice/search, separate from docs/animals.txt. I created fruits.txt with the lines apple, banana, orange and Apple pie.

grep apple printed only apple. Apple pie is a different line, because the search is case-sensitive. grep -i apple printed both lines. grep -n apple added the line number, and grep -ni apple did both, so the matches were line 1 and line 4. grep -v apple printed the lines without a lowercase apple: banana, orange and Apple pie.

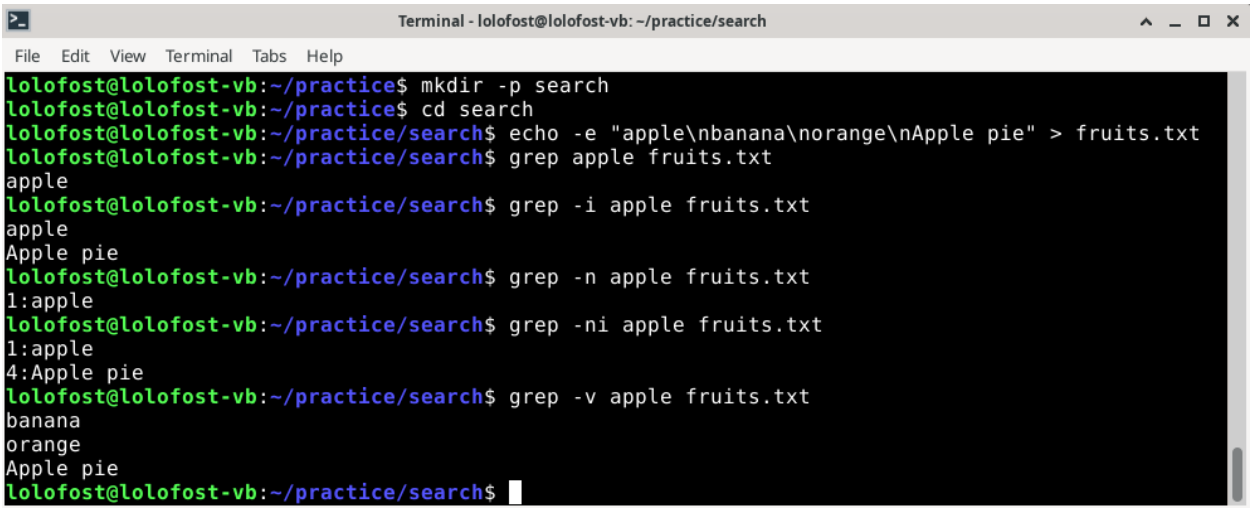


Image 7. The five grep searches.

wc -w fruits.txt printed 5. Apple pie counts as two words. wc -c fruits.txt printed 30, the byte count. The numbered grep had already shown that the file has four lines.

I created animals.txt here and sent it through a pipe. cat animals.txt | grep cat printed cat and catfish, because the letters cat are inside catfish. cat animals.txt | wc -l printed 5. cat animals.txt | sort | uniq printed the five names in alphabetical order. uniq drops a line when the same line is repeated next to it, and every name in this file is different, so the list stayed at five lines.

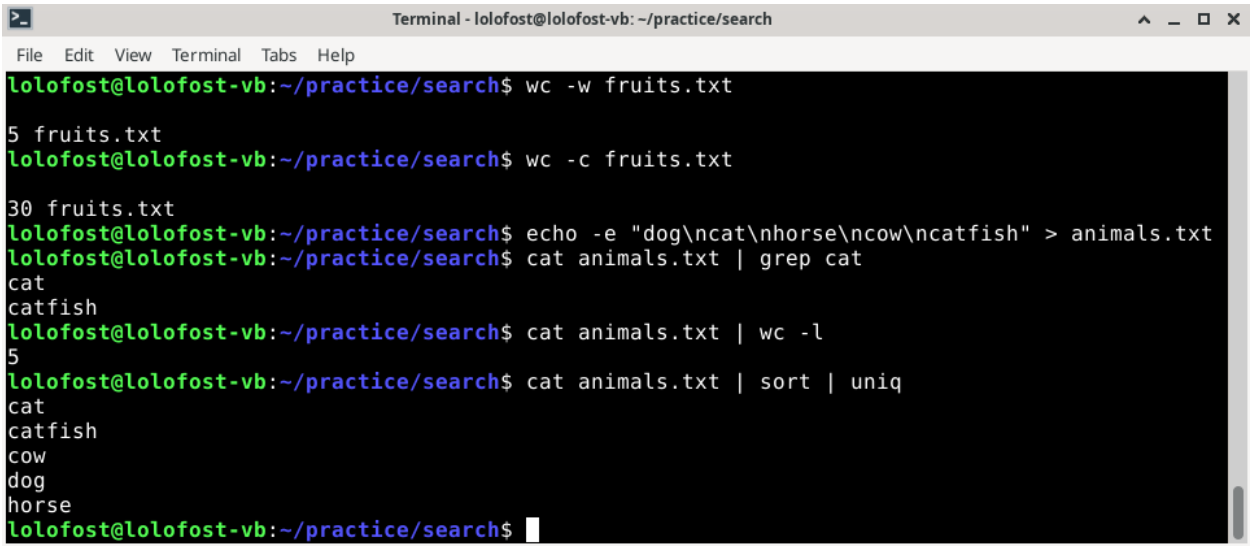


Image 8. Word count and three pipes.

GPL-2

The license on this machine is /usr/share/common-licenses/GPL-2. wc -l counted 338 lines. grep GNU prints every line that contains those letters. I passed that output through head, and the terminal showed the opening lines, including the title and the sentence

about freedom to share and change the program. grep -c GNU counted the matches instead of printing them: 8 lines.

A case-sensitive grep -c license counted 16 lines. grep -ic license ignores case and counted 56 lines. The case-insensitive search also matches License and LICENSE, so the count goes up to 56.

For my own query I combined grep, a pipe and wc. grep -i "copy" /usr/share/common-licenses/GPL-2 | wc -l counted 38 lines that contain copy in either letter case.

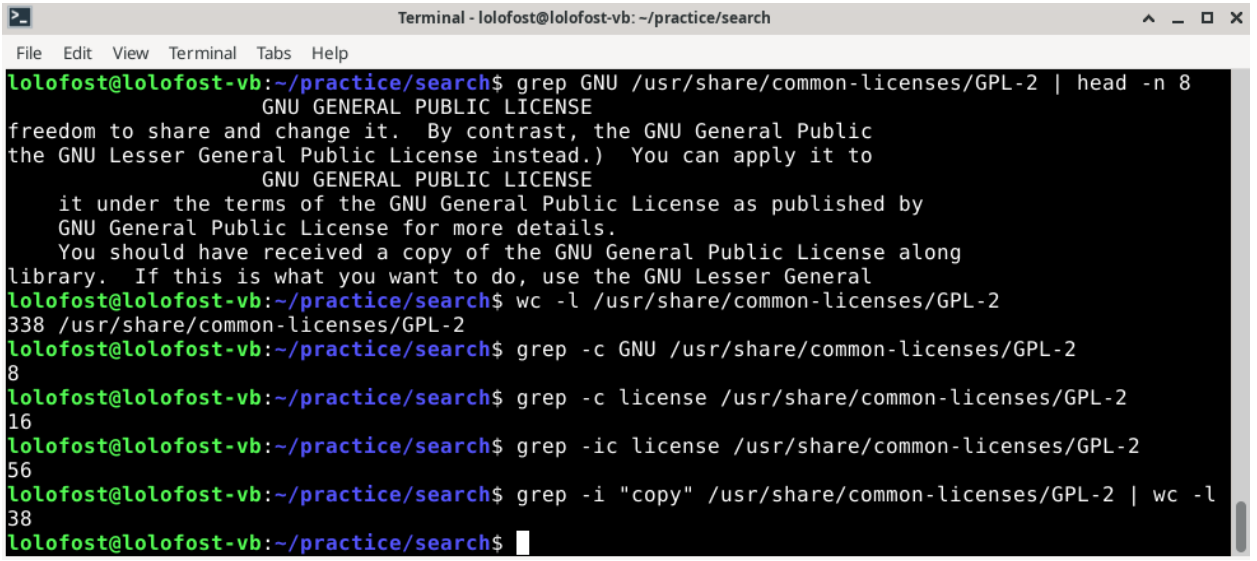


Image 9. Searches in the GPL-2 file.

GPL-2 is the license Linux itself uses. The point of it is that people may change the program and pass it on, and the source goes with the copy. The next person receives those same rights. That is what keeps a system like Linux open to changes from people other than the original authors.

btop

I installed btop with sudo apt-get install btop. which btop printed /usr/bin/btop, so the runnable file sits with the other programs under /usr/bin. dpkg -L btop listed everything the package installed: themes and other data under /usr/share/btop, icons, a manual page and a desktop file. The settings file is separate from that package list. btop created ~/.config/btop/btop.conf on the first start, and I copied it to btop.conf.orig before editing it.

I changed three values. update_ms went from 2000 to 300, so the screen refreshes faster. proc_tree went from False to True. proc_sorting went from cpu lazy to memory. On the next start the process list was a tree, sorted by memory, and the corner of the window showed 300ms. CPU wa about 1%, load average 0.06, 1.02 GiB used out of 3.81 GiB, and the root disk about half full. The network counters were at zero.

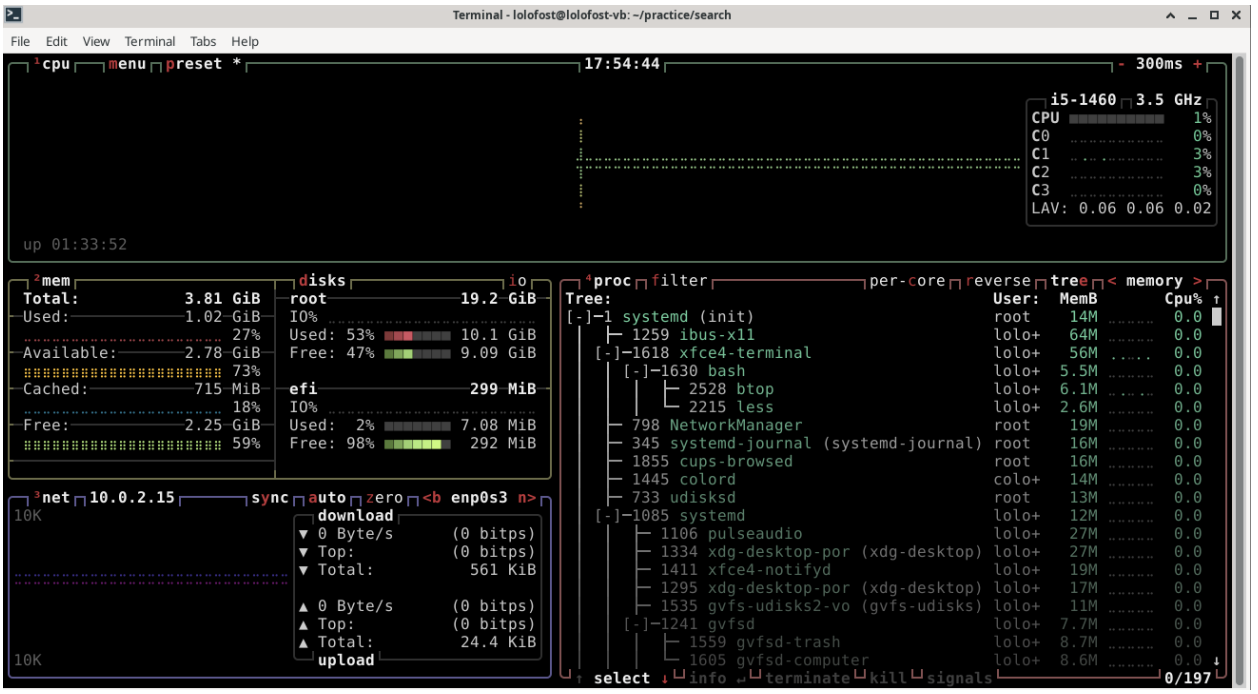


Image 10. btop after the three configuration changes.

I then set proc_tree to Truuuuue. btop dropped that value and drew a flat process list again, which is what proc_tree = False looks like. The refresh time stayed at 300ms and the list stayed sorted by memory, so the other two edits were still in effect.

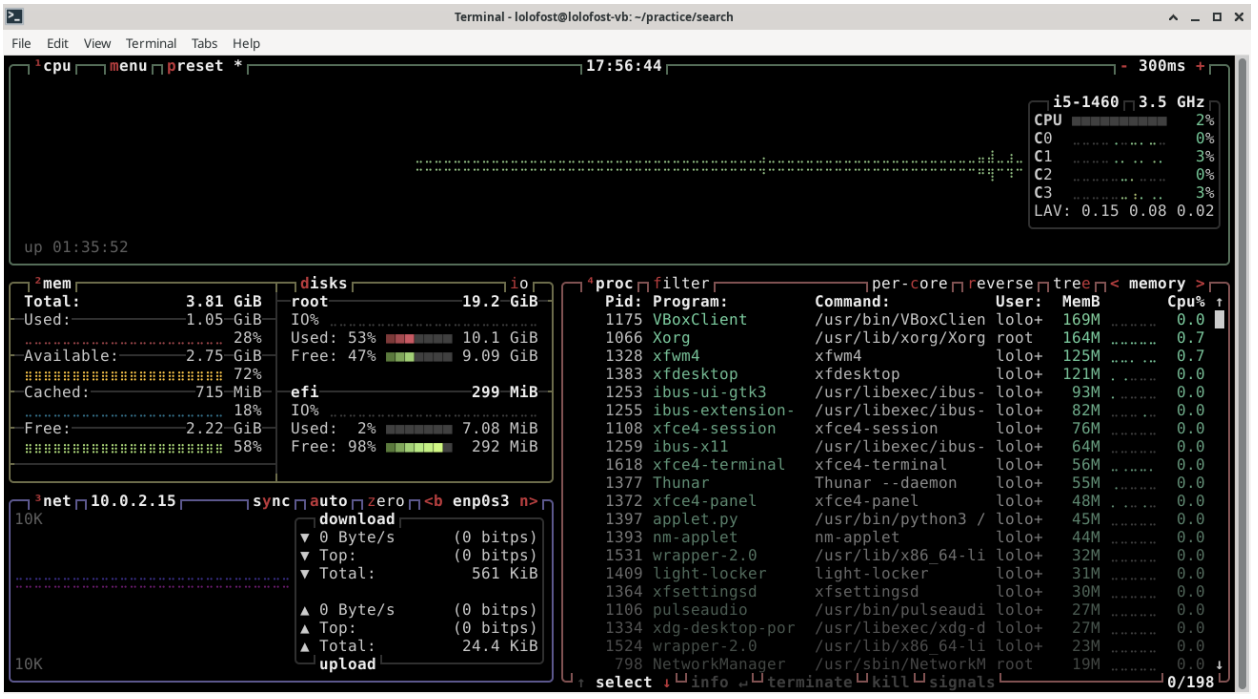


Image.11 btop after the invalid proc_tree value.

To put some traffic on the network I opened a second terminal and ran ping -i 0.1 8.8.8.8. That sends an echo request to 8.8.8.8, every tenth of a second. Replies came back at about 2 ms, and btop showed the download at roughly 925 bytes per second.

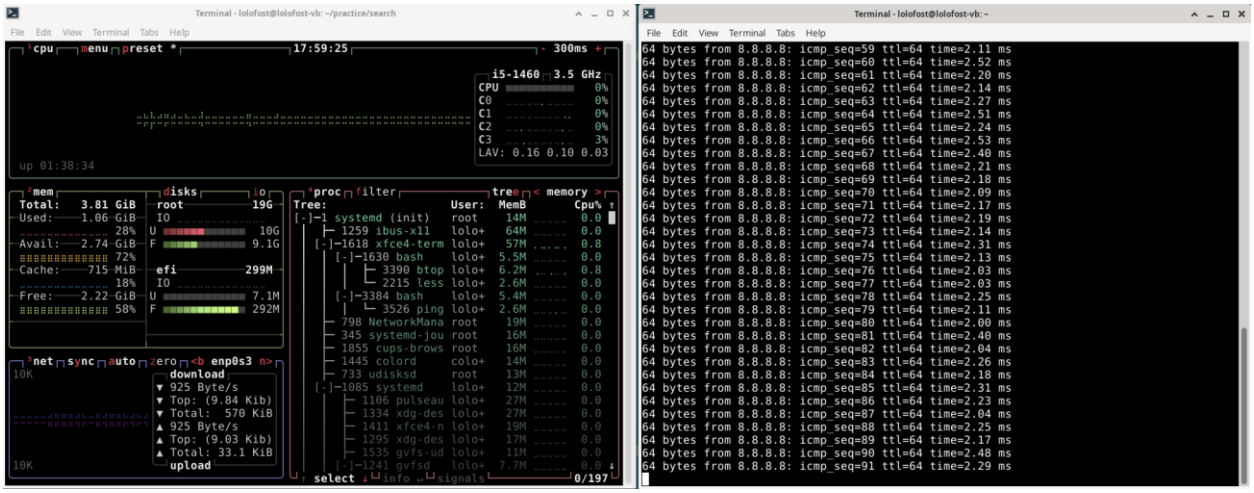


Image 12. btop while ping was running.

yes > /dev/null writes endlessly and throws the output away, which loads one core. btop showed core C1 at 100% and the yes process at about a quarter of the total CPU. The other three cores stayed almost idle. I stopped with Ctrl+C.

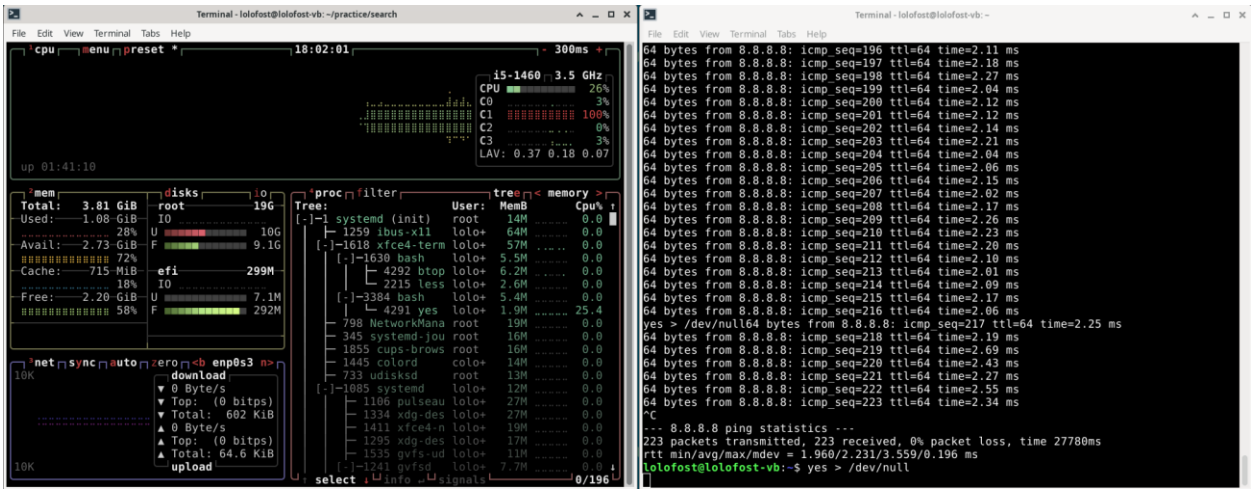


Image 13. btop while yes loaded one core.

fastfetch

I installed fastfetch and ran it with the default settings. It prints a short description of the machine.

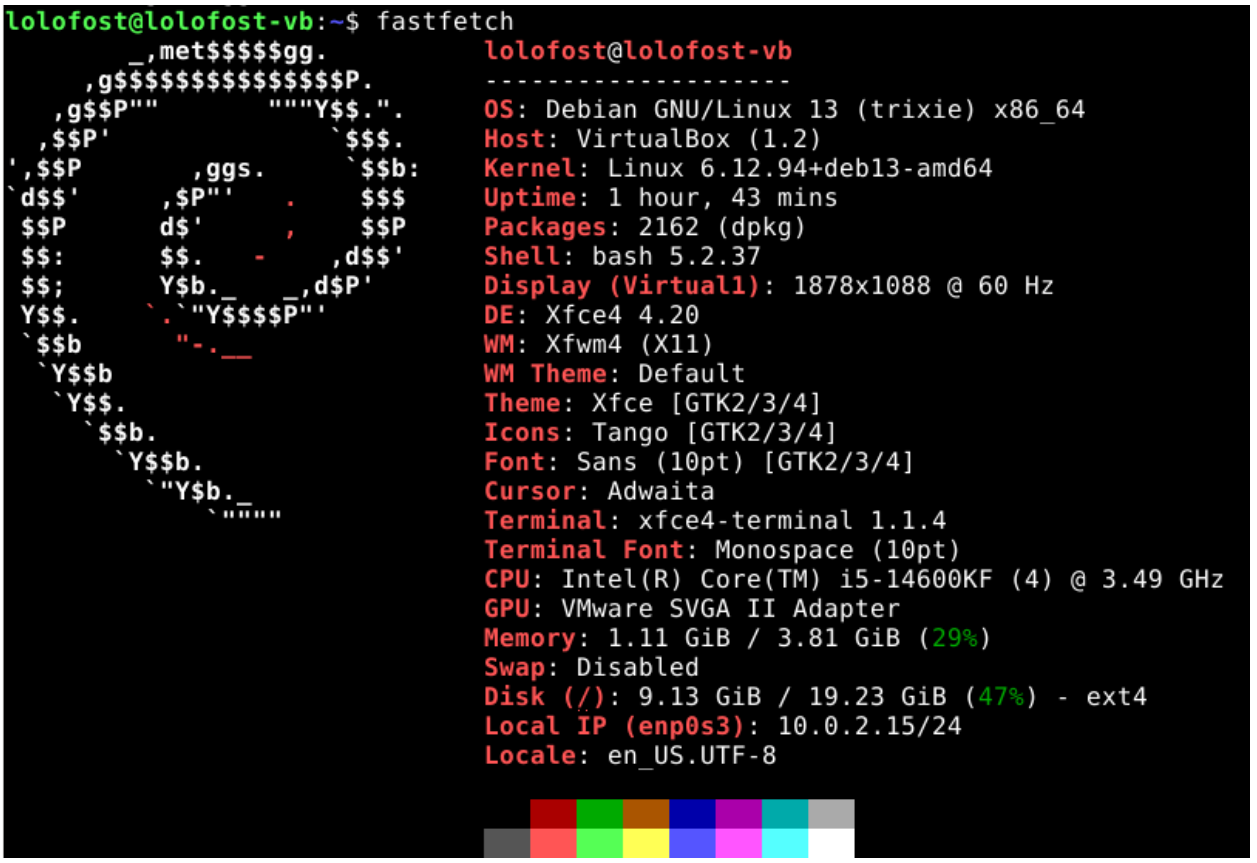


Image 14. fastfetch on the virtual machine.